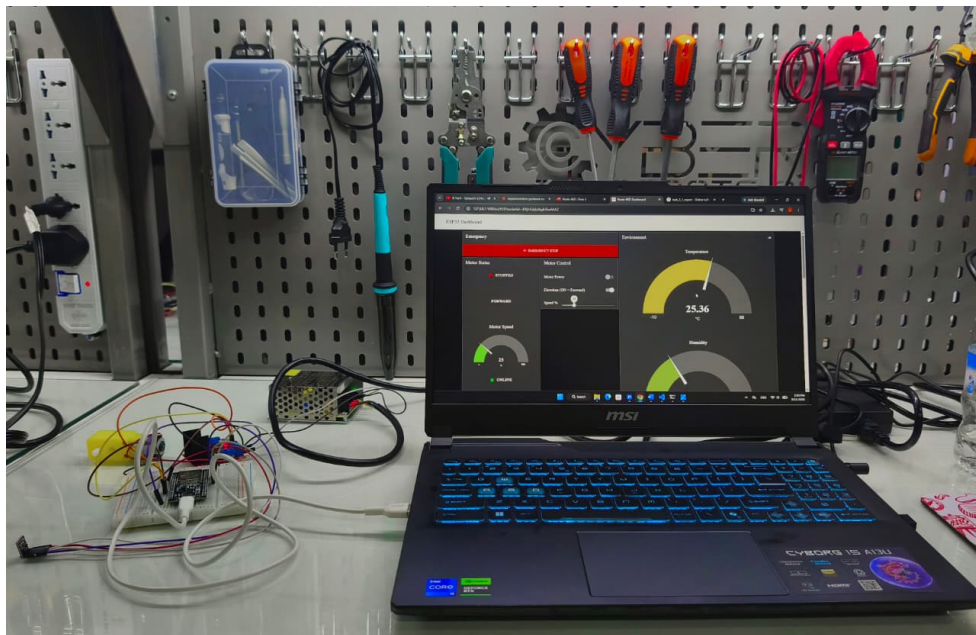


Node-RED Visual Dashboard for a Local ESP32 MQTT System

Live Sensor Telemetry, Bidirectional Motor Control, and Availability Detection over Node-RED Dashboard



*Dana Natsheh
Cyber Robot
IoT*

1. Executive Summary

This report documents Task 3.1: replacing manual MQTT Explorer/CLI inspection of the Task 2.7 ESP32 system with a live Node-RED dashboard. The dashboard runs on the same local machine as the Mosquitto broker used in Task 2.7, subscribes to the same sensor and motor topics, and adds full bidirectional control – power, direction, and speed – plus a dedicated, always-visible emergency stop.

Two firmware-side additions were made specifically to support the dashboard: a retained Last Will and Testament (LWT) on a new `device/esp32/status` topic, so the dashboard can detect an ungraceful ESP32 disconnect immediately rather than by inferring it from silence; and `retain=true` on all sensor/status publishes, so a dashboard opened after the fact still shows real current values instead of blank widgets.

The most instructive part of this task, by far, was not the dashboard layout but a two-stage debugging chase to explain why the historical line charts stayed completely blank while the live gauges worked perfectly. The eventual root cause – `mqtt in` nodes only auto-parsing JSON-shaped payloads, silently leaving a plain numeric string like "26.93" as type `string` – is not visible anywhere in the gauge or text widgets, which happily stringify anything, but is fatal to `ui_chart`, which silently refuses to plot a non-numeric `msg.payload`. A second, compounding issue (an invalid JSONata rule caused by leaving a `change` node's value-type dropdown on *number* instead of *expression*) delayed the fix further. Section walks through the full diagnostic sequence, including the false leads that were ruled out first.

2. System Overview & Goal

This task adds a presentation/control layer on top of the existing Task 2.7 hardware and broker – no ESP32 wiring, sensor, or motor driver changed:

```
(BME280, I2C) --> [ESP32] --publish--> [Mosquitto 192.168.1.33:1883] --subscribe--> [Node-RED :1880/ui]
                                     Node-RED dashboard
--publish--> [Mosquitto 192.168.1.33:1883] --subscribe--> [ESP32] --drives--> (L298N + DC Motor)
                                     ESP32
--LWT (retained, on disconnect)--> [Mosquitto] --> device/esp32/status --> [Node-RED]
```

- Node-RED runs on the same PC that hosts Mosquitto, connecting to it as just another MQTT client – no second broker instance, no change to the broker's listener/firewall setup from Task 2.7.
- The dashboard is a pure MQTT client: every value it shows is read from the broker, and every command it issues is a normal publish to the same `motor/command/*` topics the ESP32 already subscribed to.
- The ESP32 firmware gained one new topic (`device/esp32/status`) and retained delivery on its existing `telemetry/status` topics; the command topics remain deliberately non-retained.

3. Hardware Architecture & Pin Mapping

Peripheral	Signal	ESP32 GPIO	Role
BME280 (I2C)	SDA	GPIO 21	I2C data line, shared bus
BME280 (I2C)	SCL	GPIO 22	I2C clock line, shared bus
Motor driver (L298N)	IN1	GPIO 26	Direction line A
Motor driver (L298N)	IN2	GPIO 27	Direction line B
Motor driver (L298N)	ENA	GPIO 25	PWM speed line

Table 1: ESP32 pin mapping (carried over unchanged from Task 2.7).

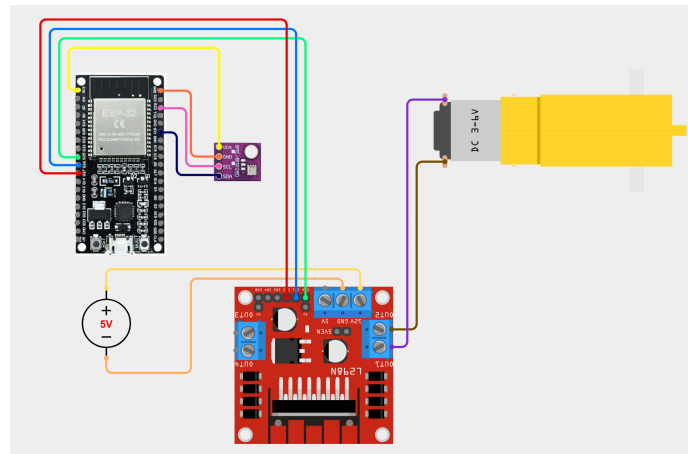


Figure 1: Wiring schematic

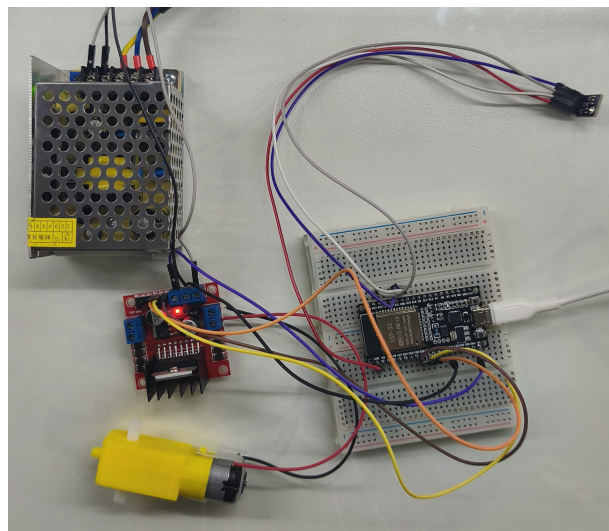


Figure 2: Assembled circuit

4. Firmware Changes for This Task

Two changes were made to the Task 2.7 firmware, both additive – no topic was renamed and no existing behavior changed.

Retained Last Will and Testament

`MqttInterlockClient::connectBroker()` now registers a will on a new topic before connecting, and publishes its own “online” the moment the connection succeeds:

```
1 connected = _mqttClient.connect(  
2     clientId.c_str(), MQTT_USERNAME, MQTT_PASSWORD,  
3     TOPIC_DEVICE_STATUS, DEVICE_STATUS_QOS, true, DEVICE_STATUS_OFFLINE);  
4  
5 if (connected) {  
6     _mqttClient.publish(TOPIC_DEVICE_STATUS, DEVICE_STATUS_ONLINE, true);  
7     resubscribeAll();  
8     _reconnectFlag = true;  
9 }
```

Listing 1: `MqttInterlockClient.cpp` – LWT registration

Because the will is retained, Mosquitto publishes “offline” on the ESP32’s behalf the instant an ungraceful disconnect is detected (crash, power loss, Wi-Fi drop) – Node-RED does not need to infer this from silence, and a dashboard opened *after* the disconnect still sees “offline” immediately because retained messages replay to new subscribers.

Retained telemetry and status publishes

All six existing publish calls in `main.cpp` gained a `retain=true` argument:

```
1 mqttClient.publish(TOPIC_SENSOR_TEMPERATURE, buf, true);  
2 mqttClient.publish(TOPIC_MOTOR_STATUS_STATE, motor.isRunning() ? "1" : "0", true);
```

Listing 2: `main.cpp` – retained publishes (excerpt)

Outgoing *commands* from Node-RED (`motor/command/*`) were deliberately left non-retained: a retained command would replay itself to the ESP32 on every reconnect or reboot, which could unexpectedly restart the motor with a stale setpoint.

5. Node-RED Dashboard Design

The dashboard is a single tab, **ESP32 Dashboard**, split into four groups:

Group	Contents
Emergency	Always-visible, full-width red EMERGENCY STOP button, wired directly to <code>motor/command/state</code> , bypassing the normal switch/clamp path entirely.
Environment	Gauge <i>and</i> 3-minute line chart for each of temperature, humidity, pressure.
Motor Status	Color-coded running/stopped indicator, direction indicator, live speed gauge & numeric readout, and a combined connection-status indicator.
Motor Control	Power switch, direction switch, and a speed slider (publishes on release, not while dragging) whose output is always routed through a clamping function before publish.

Connection monitor: LWT plus a watchdog

A single function node combines two independent signals so the indicator cannot get stuck showing stale “online” state:

```

1  const STALE_MS = 8000;
2  if (msg.topic === 'device/esp32/status') {
3      ctx.online = (msg.payload === 'online');
4      if (ctx.online) ctx.lastSeen = Date.now();
5  } else if (msg.topic !== 'tick') {
6      ctx.lastSeen = Date.now(); // any telemetry proves it's alive
7  }
8  const stale = (Date.now() - ctx.lastSeen) > STALE_MS;

```

Listing 3: Connection monitor function node (excerpt)

A 3-second repeating `inject` node re-evaluates this on a timer, so the indicator degrades from ONLINE (green) to STALE (orange) even if the broker-level LWT hasn’t fired yet – for example during a Wi-Fi hiccup that doesn’t fully drop the TCP session – and to OFFLINE (red) once the retained LWT message actually arrives.

Speed clamping, mirrored from the firmware

The same clamp-at-the-boundary philosophy used in `MotorController::setSpeedPercent()` (Task 2.7) is repeated on the Node-RED side, independently:

```

1  let v = Number(msg.payload);
2  if (isNaN(v)) v = 0;
3  if (v < 0) v = 0; else if (v > 100) v = 100;
4  msg.payload = String(Math.round(v));

```

Listing 4: Clamp speed 0-100 function node

This was verified by wiring a temporary `inject` node with payloads 150 and -40 directly into the clamp function: both were correctly forced to 100 and 0 before reaching `motor/command/speed`. Even if this Node-RED clamp were ever bypassed, the ESP32’s own clamp in `MotorController` still protects the motor – defense in depth across both layers.

6. MQTT Topic Table

Topic	Direction	Payload
sensor/bme280/temperature	ESP32 → broker	Float °C, retained
sensor/bme280/humidity	ESP32 → broker	Float %RH, retained
sensor/bme280/pressure	ESP32 → broker	Float hPa, retained
motor/status/state	ESP32 → broker	"1"/"0", retained
motor/status/direction	ESP32 → broker	"forward"/"reverse", retained
motor/status/speed	ESP32 → broker	Integer 0–100, retained
motor/command/state	Node-RED → ESP32	"1"/"0"
motor/command/direction	Node-RED → ESP32	"forward"/"reverse"
motor/command/speed	Node-RED → ESP32	Integer, clamped 0–100
device/esp32/status <i>(new)</i>	ESP32 → broker	"online"/"offline", retained, LWT

Table 2: Full topic namespace; the last row is new for this task.

7. Debugging Journal: The Blank-Chart Investigation

This section is included in full because ruling out the wrong causes first was as instructive as finding the real one – and mirrors the layer-isolation debugging discipline used for the network issues in Task 2.7.

Symptom

After first import and deploy, every `mqtt in` node showed green/connected, and all three gauges (temperature, humidity, pressure) updated live and correctly. The three history charts in the same group, however, stayed completely blank: correct axis range, correct title, correct legend label – just no line, ever.

Hypothesis 1 (ruled out): widget disabled

The chart node's **Enabled** toggle was checked first, since a disabled widget renders its shell but never draws data. It was already enabled – ruled out.

Hypothesis 2 (partially true, not the root cause): buffer reset on every deploy

Each Node-RED deploy restarts the chart node and clears its in-memory point buffer, and several deploys happened in quick succession while debugging. This meant every check was, in effect, looking at a chart with only seconds of possible history – worth ruling out by waiting untouched for 30-60 seconds – but it did not explain why the line still never appeared even after a long, undisturbed wait.

Hypothesis 3 (the real root cause): payload type

Wiring a debug node directly to an `mqtt in` node's output showed the actual problem:

```

1 sensor/bme280/pressure : msg.payload : string[5]
2 "894.35"
```

Listing 5: Debug sidebar output – the payload is a string

The ESP32 sends plain ASCII numerals (`dtostrf()` output, not JSON). Node-RED's `mqtt in` node with `datatype: auto` only attempts to JSON-parse a payload that looks like a JSON object or array (starting with `{` or `[`); a bare number string is left as-is. `ui_gauge` and `ui_text` widgets do not care – they display whatever they are given – but `ui_chart` silently discards any point whose `msg.payload` is not of type `number`. No error is logged anywhere; the chart just never grows.

First fix attempt (also failed, different bug): wrong field type in the change node

A `change` node was inserted between each `mqtt in` node and its gauge/chart pair, set to `Set msg.payload to the value $number(payload)`. Deploying immediately produced red warning triangles on all three nodes:

```
1 The workspace contains some nodes that are not properly configured:
2 - [Task 3.1 - ESP32 Dashboard] string to int (change) x3
```

Listing 6: Deploy warning

The cause was a second, independent slip: the value field's type selector had been left on `number` instead of `expression (JSONata)`. With the field typed as a literal number, Node-RED tried to parse the text `$number(payload)` as a number itself – which fails – rather than evaluating it as a JSONata expression. Switching that one dropdown to `expression` resolved the invalid-node warning immediately.

Confirmation

With the field type corrected, the same debug node (now wired after the change node) showed the fix working:

```
1 sensor/bme280/temperature : msg.payload : number
2 26.73
```

Listing 7: Debug sidebar output – fixed

Result

With correctly-typed numeric data flowing continuously and a full, undisturbed wait after the last deploy, the charts began drawing correctly, including live tooltips on hover (Figure ??).

Takeaway: a widget that silently drops bad data is far harder to debug than one that errors loudly. The fix here was not a Node-RED bug at all – it was an implicit type-coercion boundary (`auto` datatype parsing) combined with an ESP32 firmware choice (plain ASCII payloads, not JSON) that only became visible once a debug node was inserted *directly* on the raw `mqtt in` output rather than trusted from further downstream.

8. Results & Verification

Startup and subscription

The Node-RED broker connection log shows the expected reconnect pattern: several rapid retries while the network/broker were still settling, followed by a successful connection – confirming the existing

rate-limited reconnect behavior (carried over conceptually from the ESP32 side's own reconnect logic in Task 2.7) is exactly as expected on the Node-RED side too.

```
11 Aug 11:00:44 - [info] Starting flows
11 Aug 11:00:44 - [info] Started flows
11 Aug 11:00:46 - [info] [mqtt-broker:Local Mosquitto ] Connection failed to
broker: nodered-task31-dashboard@mqtt://192.168.1.33:1883
11 Aug 11:01:04 - [info] [mqtt-broker:Local Mosquitto ] Connection failed to
broker: nodered-task31-dashboard@mqtt://192.168.1.33:1883
11 Aug 11:01:21 - [info] [mqtt-broker:Local Mosquitto ] Connection failed to
broker: nodered-task31-dashboard@mqtt://192.168.1.33:1883
11 Aug 11:01:38 - [info] [mqtt-broker:Local Mosquitto ] Connection failed to
broker: nodered-task31-dashboard@mqtt://192.168.1.33:1883
11 Aug 11:01:53 - [info] [mqtt-broker:Local Mosquitto ] Connected to broker:
nodered-task31-dashboard@mqtt://192.168.1.33:1883
```

Figure 3: Node-RED startup log: repeated connection attempts followed by success.

Mosquitto's own verbose log confirms every expected topic was subscribed to on connect, including the new LWT status topic:

```
1786435313: nodered-task31-dashboard 1 motor/status/direction
1786435313: Sending SUBACK to nodered-task31-dashboard
1786435313: Received SUBSCRIBE from nodered-task31-dashboard
1786435313:      motor/status/speed (QoS 1)
1786435313: nodered-task31-dashboard 1 motor/status/speed
1786435313: Sending SUBACK to nodered-task31-dashboard
1786435313: Received SUBSCRIBE from nodered-task31-dashboard
1786435313:      device/esp32/status (QoS 1)
1786435313: nodered-task31-dashboard 1 device/esp32/status
1786435313: Sending SUBACK to nodered-task31-dashboard
```

Figure 4: Mosquitto log showing the Node-RED client subscribing to all seven topics, including device/esp32/status.

Offline state, verified before the ESP32 was reachable

Before the ESP32 was reflashed/connected, the dashboard correctly showed **OFFLINE** with zeroed/blank widgets rather than stale or misleading data:

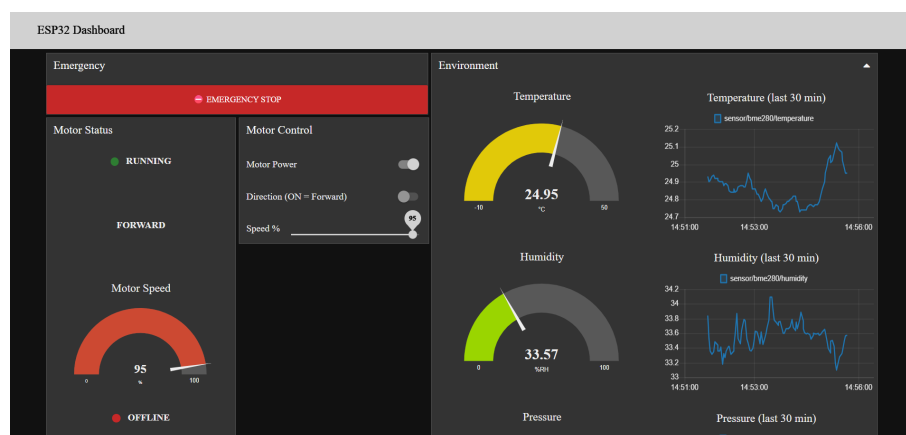


Figure 5: Dashboard before the ESP32 connected: OFFLINE indicator, zeroed speed gauge, no telemetry yet.

Live control and gauges

With the ESP32 connected and the speed slider set to a non-zero value, the motor ran forward at the commanded speed, and the dashboard's Motor Status group reflected the ESP32's real reported state (not an echo of the last command) – direction FORWARD, speed 23%, connection ONLINE:

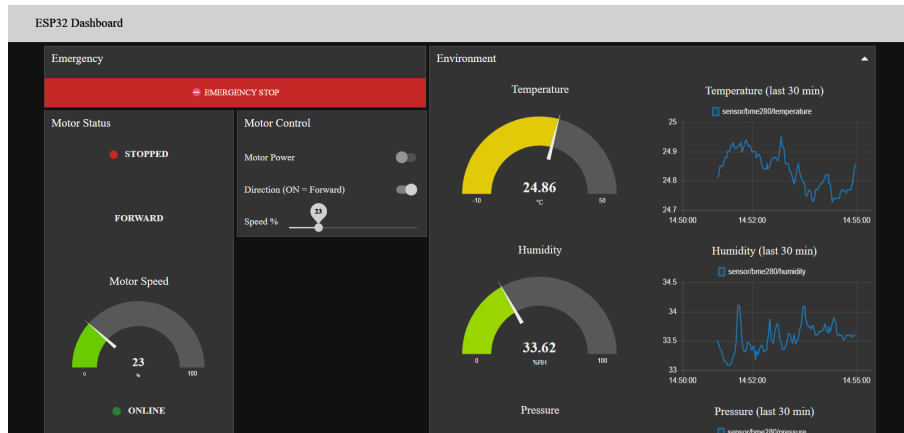


Figure 6: Motor forward at 23% speed; connection status ONLINE, Temperature, humidity, and pressure gauges updating live from real BME280 readings..

Live charts

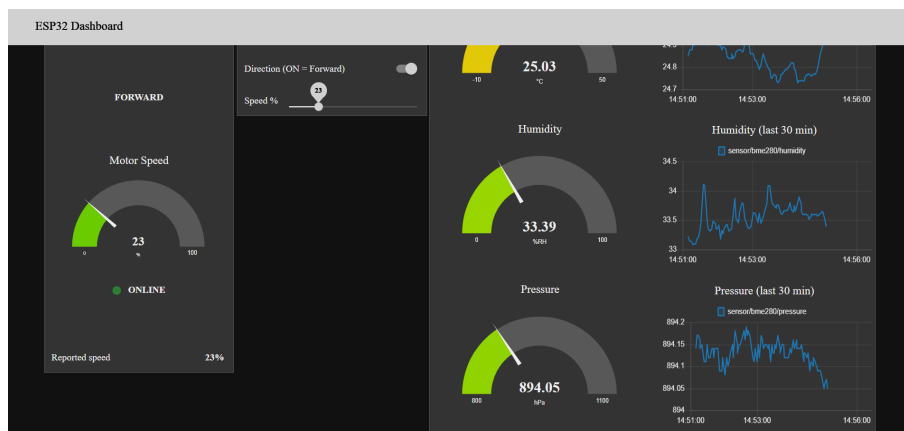


Figure 7: Humidity and pressure history charts drawing correctly, with a live hover tooltip showing an exact retained value at a specific timestamp.

Full dashboard, zoomed out

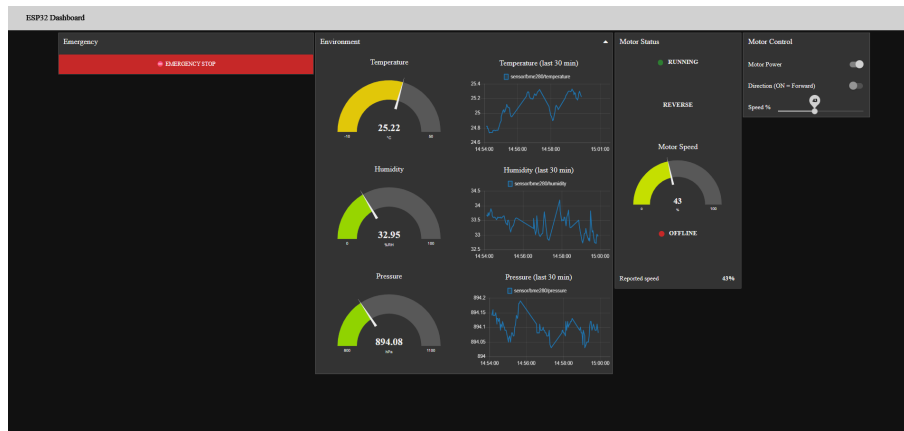


Figure 8: All four groups visible together at reduced browser zoom: Emergency, Environment, Motor Status, Motor Control.

9. Design Notes – Efficient Choices

1. **Dual-signal availability detection.** Combining a broker-guaranteed LWT with a client-side watchdog timer means the indicator degrades gracefully (ONLINE → STALE → OFFLINE) instead of only ever having two states that can each, independently, lag reality.
2. **Retained telemetry, non-retained commands.** A single, deliberate asymmetry: state the world should always be able to see on connect (sensor values, motor status, availability) is retained; instructions that should never silently replay (motor commands) are not.
3. **Clamping at two independent boundaries.** The Node-RED-side clamp function and the ESP32-side `setSpeedPercent()` clamp are not redundant by accident – either one failing (a Node-RED flow bug, or a future code path that calls `setSpeedPercent()` directly) still leaves the other in place.
4. **Debugging by isolating the exact stage.** Wiring a debug node directly onto a raw `mqtt in` output – rather than trusting a downstream widget's behavior – cleanly separated “is data arriving” from “is data the right type” from “is the widget rendering correctly”, which is what actually found the real root cause after two plausible-but-wrong hypotheses.
5. **Reused, not duplicated, firmware philosophy.** The LWT and retained-publish additions extend Task 2.7's existing publish/subscribe wrapper rather than introducing a parallel mechanism – both were listed as explicit *Future Improvements* in the Task 2.7 report and are implemented here exactly as scoped there.

10. Known Limitations & Considerations

- **Software-only emergency stop.** The EMERGENCY STOP button publishes to the same `motor/command/state` topic as the normal power switch; it is a separate, always-visible UI control for the fastest possible *software* stop, but it still depends on Wi-Fi/MQTT delivery and the ESP32 being responsive – it is not a substitute for a physical kill switch or relay.
- **Chart history does not survive a Node-RED restart.** `ui_chart` buffers its points in memory on the Node-RED process; a restart (not just a redeploy of this one flow) clears it, since Task 3.1 only requires persistence across a session, not across restarts.
- **Broker credentials are not preserved on flow export.** `flows.json` must have the MQTT broker's username/password re-entered after import, since Node-RED intentionally strips credential fields from exported flows.
- **Unencrypted transport, inherited from Task 2.7.** Both the ESP32↔broker and Node-RED↔broker links remain plain MQTT on port 1883, not TLS.
- **Dashboard itself has no authentication configured.** Anyone who can reach `:1880/ui` on the LAN can control the motor; this was out of scope for this task but is a real gap for anything beyond a bench setup.

11. Bill of Materials

Item	Qty	Notes
ESP32 DevKit development board	1	Task 2.7 firmware plus LWT/retained-publish additions
BME280 environmental sensor module	1	I2C, address 0x76
L298N (or equivalent) H-bridge motor driver	1	Driven via IN1/IN2/ENA
DC motor	1	Bidirectional, speed-controlled via PWM
Breadboard + jumper wires	1 set	–
PC running Mosquitto 2.1.2	1	Same broker host as Task 2.7
PC running Node.js + Node-RED + node-red-dashboard	1	Same machine as the broker in this setup
Web browser	1	Dashboard client at <code>:1880/ui</code>

12. Future Improvements

- **Persist chart history** to a time-series store (e.g. InfluxDB) so trends survive a Node-RED restart, not just a session.
- **Dashboard authentication** via Node-RED's `adminAuth/httpNodeAuth` settings, so the control surface isn't open to anyone on the LAN.
- **TLS** on both the ESP32↔broker and Node-RED↔broker links.
- **A real hardware e-stop** (a relay cutting motor power directly, independent of Wi-Fi/MQTT) to complement the current software-only emergency stop button.
- **Migrate to node-red-dashboard 2.0** for a more modern widget set and better default responsive behavior, once the classic-dashboard layout quirks encountered in Section become a bigger constraint than the migration cost.